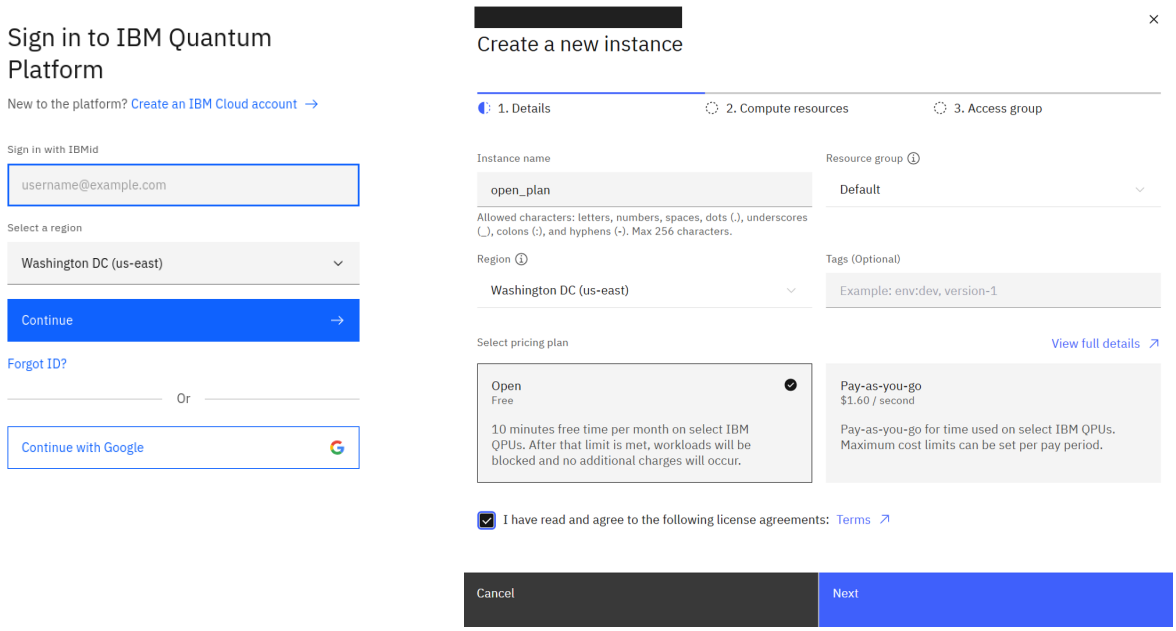
01 Account, instance and API key

IBM setup is needed for live calibration access and physical QPU use. **Local simulations with bundled or uploaded calibration JSON do not need an IBM account.**

1. Sign in and create or select an instance

Open quantum.cloud.ibm.com/signin. Sign in with your IBMid or create an IBM Cloud account and complete verification. Check the selected Cloud account. For a free Open Plan instance, choose **Washington DC (us-east)**. [1, 2]

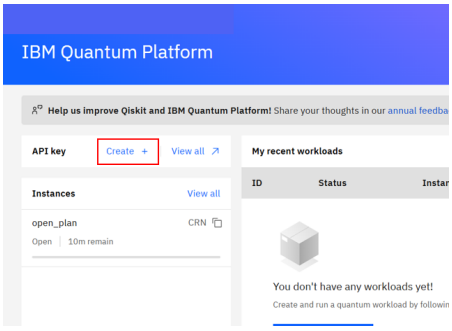
Open **Instances > Create instance** (or use an existing suitable instance). Choose a name such as **quantivex-open** and the **Open** plan; review the terms. Continue through compute resources and access group, then create the instance. Return to Instances and copy its **CRN**. [2]



Left: public sign-in page from the supplied guide. Right: historical instance-creation reference (Vašina & Friák, 2026, Fig. 7). Follow current account terms; screenshot labels and QPU names can differ.

2. Create an API key

On **Home**, find the API-key section and choose **Create API key / Create +**. Name the key, create it and copy it immediately: IBM does not display its value again. The key authenticates software; the CRN selects your instance. [3]



Historical dashboard crop: API-key entry point (Vašina & Friák, 2026, Fig. 10). The creation dialog is not shown. No secret key is displayed.

Open Plan: standard allowance is up to 10 QPU minutes per rolling 28-day window; check your dashboard for current terms and promotions. Keep the API key and **~/.qiskit/qiskit-ibm.json** private. Revoke a key if exposed. [2, 3]

02 Install, save credentials and verify

Use a trusted personal computer. Commands below use **Bash in Linux, macOS or WSL**. Start inside the extracted **quantivex/** folder containing pyproject.toml.

3. Install Quantivex in a virtual environment

```
python3.12 -m venv .venv
source .venv/bin/activate
python -m pip install ".[gui,ibm]"
quantivex version
quantivex doctor --provider ollama
```

Python 3.12 is a suggested choice; v1.2.2 declares Python >=3.10. The **ibm** extra pins qiskit-ibm-runtime >=0.50,<0.51. Doctor checks local dependencies/configuration; it does not test IBM access or contact the model.

Windows PowerShell: use **py -3.12 -m venv .venv**, then **.venv\Scripts\Activate.ps1**; the remaining Python/Quantivex commands are the same. Native Windows lacks PySCF wheels; bundled integrals cover standard geometries.

4. Save the API key and CRN once

Save this code as **connect_ibm.py** in a private folder, then run **python connect_ibm.py** in the activated environment. Enter the key at the hidden prompt. Run again only to replace this named account. [3]

```
from getpass import getpass
from qiskit_ibm_runtime import QiskitRuntimeService

QiskitRuntimeService.save_account(
    channel="ibm_quantum_platform",
    token=getpass("IBM Quantum Platform API key: "),
    instance=input("IBM Quantum instance CRN: ").strip(),
    name="quantivex-open",
    overwrite=True,
)
print("Saved IBM account as quantivex-open.")
```

5. Verify access without submitting a job

Save this separately as **check_ibm.py**, then run **python check_ibm.py**. It loads the same account and lists accessible operational QPUs. [4]

```
from qiskit_ibm_runtime import QiskitRuntimeService

service = QiskitRuntimeService(name="quantivex-open")
backends = service.backends(operational=True, simulator=False)
print("IBM connection successful.")
print("Accessible QPUs:", [b.name for b in backends])
```

Expected: a connection message and backend-name list. Authentication failure or no accessible backends requires checking the key, account, CRN, instance permissions and backend availability. Saving credentials alone does not verify access.

Validation boundary: command arguments were checked against v1.2.2; the Python blocks passed syntax and mocked-call checks. Installation, live IBM authentication and QPU execution were not run in this environment.

03 Connect Quantivex and run locally

The saved IBM account and the LLM provider are separate configurations. An IBM API key is **not** an OpenAI, Anthropic or Ollama key.

6. Select the saved IBM account

```
unset QUANTIVEX_IBM_TOKEN QUANTIVEX_IBM_INSTANCE
export QUANTIVEX_IBM_ACCOUNT=quantivex-open
```

Set these in each terminal session used for IBM operations. This selects the local saved-account name from page 2; it need not match the IBM instance name. Clearing the other two variables avoids conflicting tokens or an unintended CRN override.

PowerShell equivalent: Remove-Item Env:QUANTIVEX_IBM_TOKEN, Env:QUANTIVEX_IBM_INSTANCE -ErrorAction SilentlyContinue $env:QUANTIVEX_IBM_ACCOUNT="quantivex-open"

7. Start the GUI and configure a tool-capable model

```
quantivex-gui
```

Open the local URL printed in the terminal (normally **http://127.0.0.1:8501**). Start Ollama with a tool-capable model already installed. In the sidebar choose **Simulation assistant** and set:

Provider: Ollama (local) | **Model ID:** your exact installed model name
API base URL: http://127.0.0.1:11434/v1 | **API key:** blank for unauthenticated local Ollama
First-turn tool choice: auto | **Shots:** 4000 | **Seed:** 1234

Choose **Check LLM compatibility**. Then choose **Bundled historical example** (no IBM login needed) or **Upload JSON**. Paste this into **Research question** and select **Ask Quantivex**:

> **Run H2 at 0.75 angstrom with active space 2e2o, compact circuit mode, best_chain layout and readout mitigation enabled. Report reference and noisy energies, selected qubits, sampling uncertainty and verdict. State that this is a local simulation.**

Review tool evidence and download the report and complete JSON record. The LLM selects tools and explains results; Quantivex/Python perform calculations. Compact mode uses classically calculated state preparation. Sampling bounds do not cover calibration drift or all modelling error.

8. Optional: export current calibration; hardware is separate

Replace **ibm_fez** with a backend listed by your connection check. This downloads calibration data and does not submit a QPU job. Upload the JSON into the GUI afterward.

```
quantivex export --backend ibm_fez \
  --output ibm_current_calibration.json
```

The GUI IBM workspace supports offline fake-backend testing. For physical QPUs, follow **docs/IBM_EXECUTION.md**: prepare, inspect, explicitly submit, then fetch. Consult **quantivex ibm submit --help** for required confirmation and total-shot budget. Controlled multi-file studies use the Python API/script.

References and troubleshooting

401/403: check key, instance and permissions. LLM errors: verify the local server, exact model ID and tool support. Linux static-TLS import errors: use the release troubleshooting instructions; the GLIBC workaround is environment-specific.

1. IBM account setup
2. IBM plans and Open Plan terms
3. Save credentials and copy CRN
4. Initialize the service

Source checks: v1.2.2 pyproject.toml; cli.py; ibm_execution.py; ibm_import.py; app.py; launcher.py; docs/LLM_PROVIDERS.md. Screenshot credits retained from the supplied guide; authenticated views are historical references, not a newly tested account walkthrough.